```
!pip -q install rdkit

import os
import random
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from rdkit import Chem, RDLogger
from rdkit.Chem import AllChem
RDLogger.DisableLog('rdApp.*')

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve
)
from sklearn.preprocessing import StandardScaler
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36.7/36.7 MB 36.0 MB/s eta 0:00:00
```

```
SEED = 42
N_SPLITS = 5
VAL_SIZE_WITHIN_TEMP = 0.5  # temp set is split equally into val and test
RADIUS = 2
N_BITS = 2048

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

set_seed(SEED)

def smiles_to_morgan_fp(smiles_list, radius=2, n_bits=2048):
    fps = []
    for smi in smiles_list:
        mol = Chem.MolFromSmiles(smi)
        if mol is not None:
            fp = AllChem.GetMorganFingerprintAsBitVect(mol, radius, nBits=n_bits)
            fps.append(np.array(fp, dtype=np.float32))
        else:
            fps.append(np.zeros(n_bits, dtype=np.float32))
    return np.array(fps, dtype=np.float32)

def build_model(random_state):
    from sklearn.neural_network import MLPClassifier
    return MLPClassifier(
        hidden_layer_sizes=(100,),
        max_iter=200,
        random_state=random_state
    )

def maybe_scale(X_train, X_val, X_test):
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_val = scaler.transform(X_val)
    X_test = scaler.transform(X_test)
    return X_train, X_val, X_test


def evaluate_fold(model, X_test, y_test):
    y_pred = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_test)[:, 1]
    else:
        y_score = model.decision_function(X_test)

    return {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_score)
    }, y_score
```

```
df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)
```

```
X = smiles_to_morgan_fp(df["smiles"], radius=RADIUS, n_bits=N_BITS)
y = df["HIV_active"].astype(int).values

skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)

fold_results = []
roc_curves = []

for fold, (train_idx, temp_idx) in enumerate(skf.split(X, y), start=1):
    print(f"\n===== FOLD {fold} / {N_SPLITS} =====")
    set_seed(SEED + fold)

    X_train_full, y_train_full = X[train_idx], y[train_idx]
    X_temp, y_temp = X[temp_idx], y[temp_idx]

    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp,
        test_size=VAL_SIZE_WITHIN_TEMP,
        stratify=y_temp,
        random_state=SEED + fold
    )

    X_train, X_val, X_test = maybe_scale(X_train_full, X_val, X_test)

    model = build_model(random_state=SEED + fold)
    model.fit(X_train, y_train_full)

    metrics, y_score = evaluate_fold(model, X_test, y_test)
    fold_results.append({
        "fold": fold,
        "test_accuracy": metrics["accuracy"],
        "test_precision": metrics["precision"],
        "test_recall": metrics["recall"],
        "test_f1": metrics["f1"],
        "test_roc_auc": metrics["roc_auc"]
    })

    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_curves.append((fold, fpr, tpr, metrics["roc_auc"]))

    print({k: round(v, 4) if isinstance(v, float) else v for k, v in fold_results[-1].items()})

results_df = pd.DataFrame(fold_results)
results_df
```

```
===== FOLD 1 / 5 =====
{'fold': 1, 'test_accuracy': 0.8443, 'test_precision': 0.8923, 'test_recall': 0.7891, 'test_f1': 0.8375, 'test_ro

===== FOLD 2 / 5 =====
{'fold': 2, 'test_accuracy': 0.8339, 'test_precision': 0.9024, 'test_recall': 0.7551, 'test_f1': 0.8222, 'test_ro

===== FOLD 3 / 5 =====
{'fold': 3, 'test_accuracy': 0.8374, 'test_precision': 0.8633, 'test_recall': 0.8108, 'test_f1': 0.8362, 'test_ro

===== FOLD 4 / 5 =====
{'fold': 4, 'test_accuracy': 0.8651, 'test_precision': 0.8759, 'test_recall': 0.8581, 'test_f1': 0.8669, 'test_ro

===== FOLD 5 / 5 =====
{'fold': 5, 'test_accuracy': 0.8201, 'test_precision': 0.8636, 'test_recall': 0.7703, 'test_f1': 0.8143, 'test_ro
```

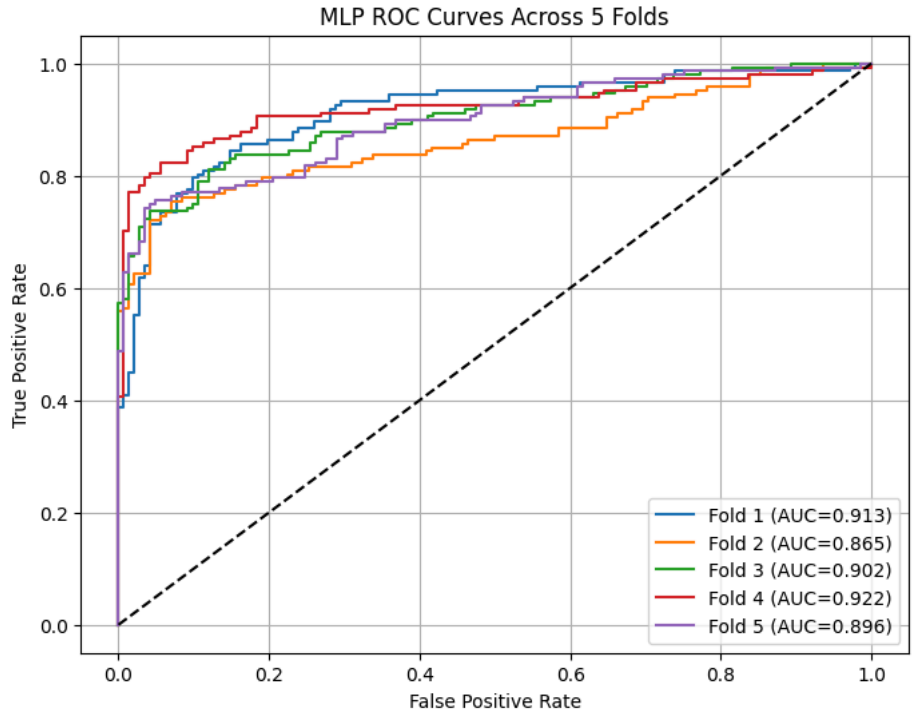
	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
0	1	0.844291	0.892308	0.789116	0.837545	0.913146
1	2	0.833910	0.902439	0.755102	0.822222	0.864664
2	3	0.837370	0.863309	0.810811	0.836237	0.902291
3	4	0.865052	0.875862	0.858108	0.866894	0.921794
4	5	0.820069	0.863636	0.770270	0.814286	0.896109

```
summary_rows = []
for metric in ["test_accuracy", "test_precision", "test_recall", "test_f1", "test_roc_auc"]:
    mean_val = results_df[metric].mean()
    std_val = results_df[metric].std(ddof=1)
    var_val = results_df[metric].var(ddof=1)
    summary_rows.append({
        "Metric": metric.replace("test_", "").upper(),
        "Mean": mean_val,
        "Std": std_val,
        "Variance": var_val,
        "Formatted": f"{mean_val:.4f} ± {std_val:.4f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.840138	0.016486	0.000272	0.8401 ± 0.0165
1	PRECISION	0.879511	0.017445	0.000304	0.8795 ± 0.0174
2	RECALL	0.796681	0.040176	0.001614	0.7967 ± 0.0402
3	F1	0.835437	0.020104	0.000404	0.8354 ± 0.0201
4	ROC_AUC	0.899601	0.021886	0.000479	0.8996 ± 0.0219

```
plt.figure(figsize=(8, 6))
for fold, fpr, tpr, auc_val in roc_curves:
    plt.plot(fpr, tpr, label=f"Fold {fold} (AUC={auc_val:.3f})")
plt.plot([0, 1], [0, 1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("MLP ROC Curves Across 5 Folds")
plt.legend()
plt.grid(True)
plt.show()
```




```
results_df.to_csv("MLP_fold_results.csv", index=False)
summary_df.to_csv("MLP_summary_results.csv", index=False)

print("Saved:")
print("- MLP_fold_results.csv")
print("- MLP_summary_results.csv")
```

```
Saved:
- MLP_fold_results.csv
- MLP_summary_results.csv
```

```
# ========================
# FINAL CELL: MLP LOG-LOSS PER FOLD
# ========================

from sklearn.metrics import log_loss
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Eğer önceki hücrelerde yoksa yeniden üret
if 'df' not in globals():
    df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
    df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)

if 'X' not in globals():
    X = smiles_to_morgan_fp(df["smiles"], radius=RADIUS, n_bits=N_BITS)

if 'y' not in globals():
```

```
    y = df["HIV_active"].astype(int).values

# Fold bazlı loss hesapla
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)

loss_rows = []

for fold, (train_idx, temp_idx) in enumerate(skf.split(X, y), start=1):
    print(f"\n===== FOLD {fold} / {N_SPLITS} =====")

    set_seed(SEED + fold)

    X_train_full, y_train_full = X[train_idx], y[train_idx]
    X_temp, y_temp = X[temp_idx], y[temp_idx]

    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp,
        test_size=VAL_SIZE_WITHIN_TEMP,
        stratify=y_temp,
        random_state=SEED + fold
    )

    X_train, X_val, X_test = maybe_scale(X_train_full, X_val, X_test)

    model = build_model(random_state=SEED + fold)
    model.fit(X_train, y_train_full)

    # Probability scores
    y_train_proba = model.predict_proba(X_train)[:, 1]
    y_val_proba   = model.predict_proba(X_val)[:, 1]
    y_test_proba  = model.predict_proba(X_test)[:, 1]

    # Log-loss
    train_loss = log_loss(y_train_full, y_train_proba)
    val_loss   = log_loss(y_val, y_val_proba)
    test_loss  = log_loss(y_test, y_test_proba)

    loss_rows.append({
        "fold": fold,
        "train_log_loss": train_loss,
        "val_log_loss": val_loss,
        "test_log_loss": test_loss
    })

    print({
        "fold": fold,
        "train_log_loss": round(train_loss, 4),
        "val_log_loss": round(val_loss, 4),
        "test_log_loss": round(test_loss, 4)
    })

loss_df = pd.DataFrame(loss_rows)
display(loss_df)

# Ortalama ve std
print("\nMean ± Std")
print(f"Train Log-Loss: {loss_df['train_log_loss'].mean():.4f} ± {loss_df['train_log_loss'].std(ddof=1):.4f}")
print(f"Val Log-Loss:   {loss_df['val_log_loss'].mean():.4f} ± {loss_df['val_log_loss'].std(ddof=1):.4f}")
print(f"Test Log-Loss:  {loss_df['test_log_loss'].mean():.4f} ± {loss_df['test_log_loss'].std(ddof=1):.4f}")

# Grafik
x = np.arange(1, len(loss_df) + 1)

plt.figure(figsize=(8, 5))
plt.plot(x, loss_df["train_log_loss"], marker='o', linewidth=2, label="Train Log-Loss")
plt.plot(x, loss_df["val_log_loss"], marker='s', linewidth=2, label="Validation Log-Loss")
plt.plot(x, loss_df["test_log_loss"], marker='^', linewidth=2, label="Test Log-Loss")


plt.xlabel("Fold")
plt.ylabel("Log-Loss")
plt.title("Train vs Validation Log-Loss Across 5 Folds")
plt.xticks(x)
plt.legend()
plt.grid(alpha=0.3)
plt.show()

plt.xlabel("Fold")
plt.ylabel("Log-Loss")
plt.title("MLP Log-Loss Across 5 Folds")
plt.xticks(x)
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
===== FOLD 1 / 5 =====
{'fold': 1, 'train_log_loss': 0.0005, 'val_log_loss': 0.9932, 'test_log_loss': 0.6159}

===== FOLD 2 / 5 =====
{'fold': 2, 'train_log_loss': 0.001, 'val_log_loss': 0.8154, 'test_log_loss': 0.73}

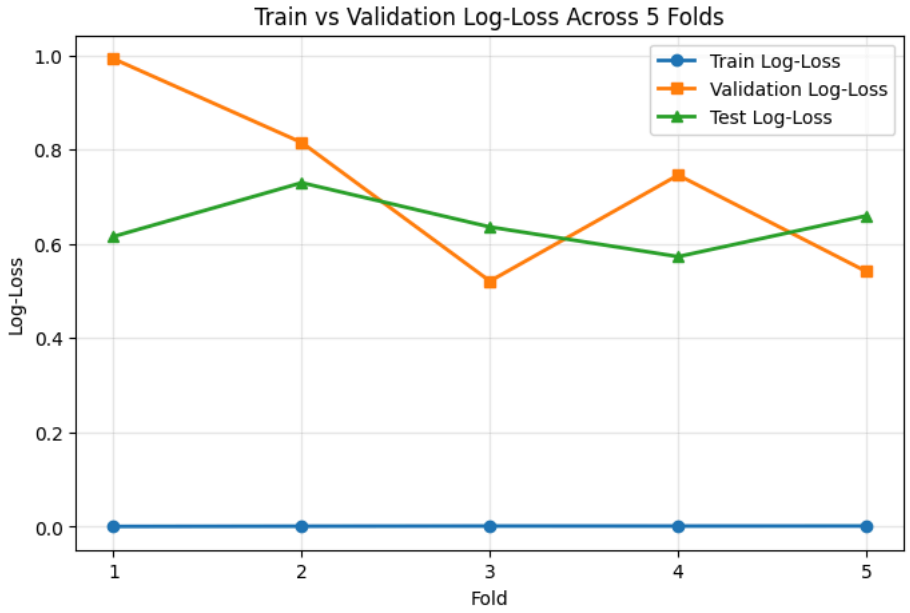
===== FOLD 3 / 5 =====
{'fold': 3, 'train_log_loss': 0.0014, 'val_log_loss': 0.5212, 'test_log_loss': 0.6361}

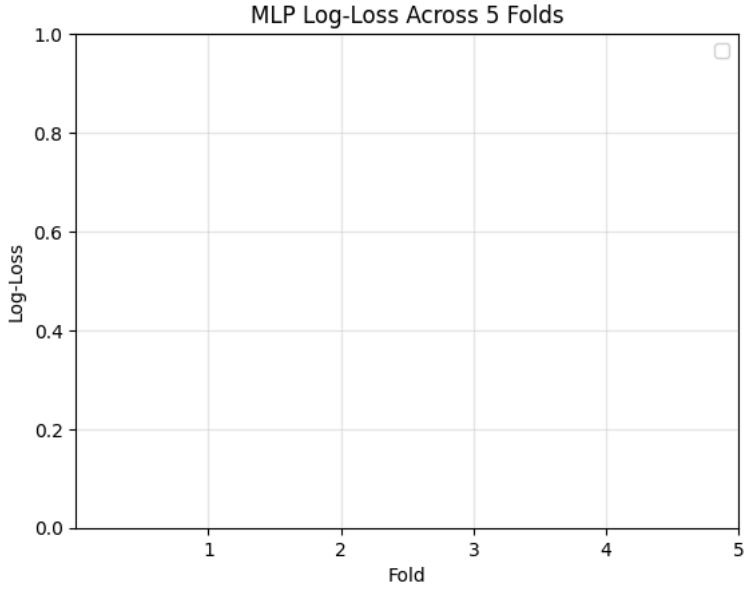
===== FOLD 4 / 5 =====
{'fold': 4, 'train_log_loss': 0.0013, 'val_log_loss': 0.7462, 'test_log_loss': 0.5732}

===== FOLD 5 / 5 =====
{'fold': 5, 'train_log_loss': 0.0014, 'val_log_loss': 0.5417, 'test_log_loss': 0.6599}
```

	fold	train_log_loss	val_log_loss	test_log_loss
0	1	0.000527	0.993150	0.615878
1	2	0.000978	0.815358	0.729991
2	3	0.001377	0.521191	0.636148
3	4	0.001309	0.746214	0.573203
4	5	0.001403	0.541668	0.659874

```
Mean ± Std
Train Log-Loss: 0.0011 ± 0.0004
Val Log-Loss:   0.7235 ± 0.1973
Test Log-Loss:  0.6430 ± 0.0581
```



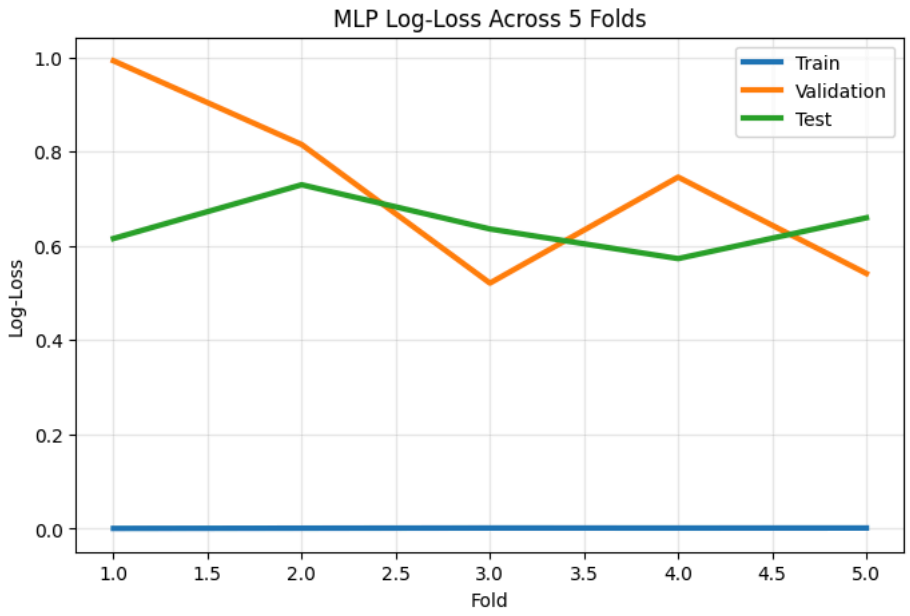



```
plt.figure(figsize=(8,5))

plt.plot(x, loss_df["train_log_loss"], linewidth=3, label="Train")
```

```
plt.plot(x, loss_df["val_log_loss"], linewidth=3, label="Validation")
plt.plot(x, loss_df["test_log_loss"], linewidth=3, label="Test")

plt.xlabel("Fold")
plt.ylabel("Log-Loss")
plt.title("MLP Log-Loss Across 5 Folds")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

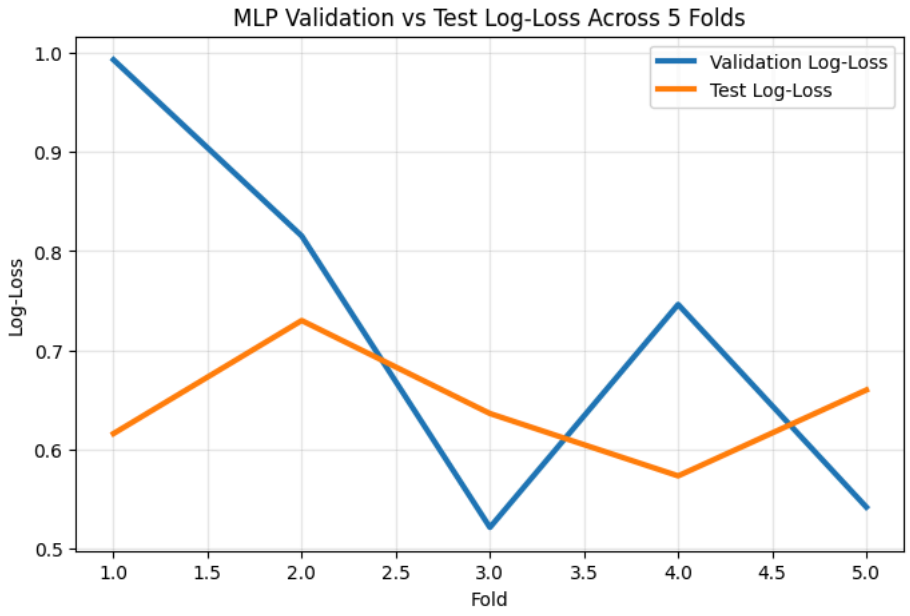



```
plt.figure(figsize=(8,5))

plt.plot(x, loss_df["val_log_loss"], linewidth=3, label="Validation Log-Loss")
plt.plot(x, loss_df["test_log_loss"], linewidth=3, label="Test Log-Loss")

plt.xlabel("Fold")
plt.ylabel("Log-Loss")
plt.title("MLP Validation vs Test Log-Loss Across 5 Folds")
plt.legend()
plt.grid(alpha=0.3)

plt.show()
```




```
plt.figure(figsize=(8,5))

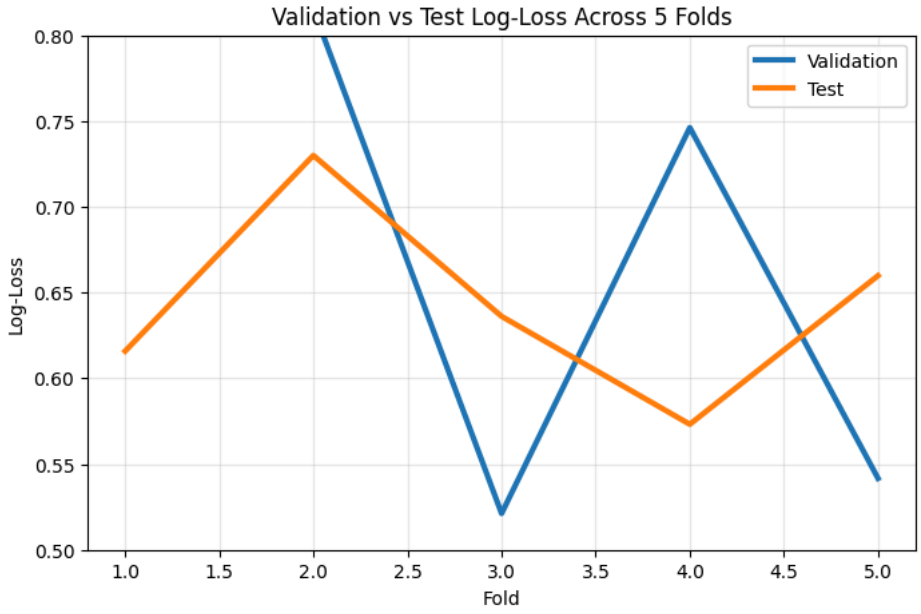
plt.plot(x, loss_df["val_log_loss"], linewidth=3, label="Validation")
plt.plot(x, loss_df["test_log_loss"], linewidth=3, label="Test")

plt.ylim(0.5, 0.8)   # 🔥 kritik

plt.xlabel("Fold")
plt.ylabel("Log-Loss")
plt.title("Validation vs Test Log-Loss Across 5 Folds")
```

```
plt.legend()
plt.grid(alpha=0.3)

plt.show()
```

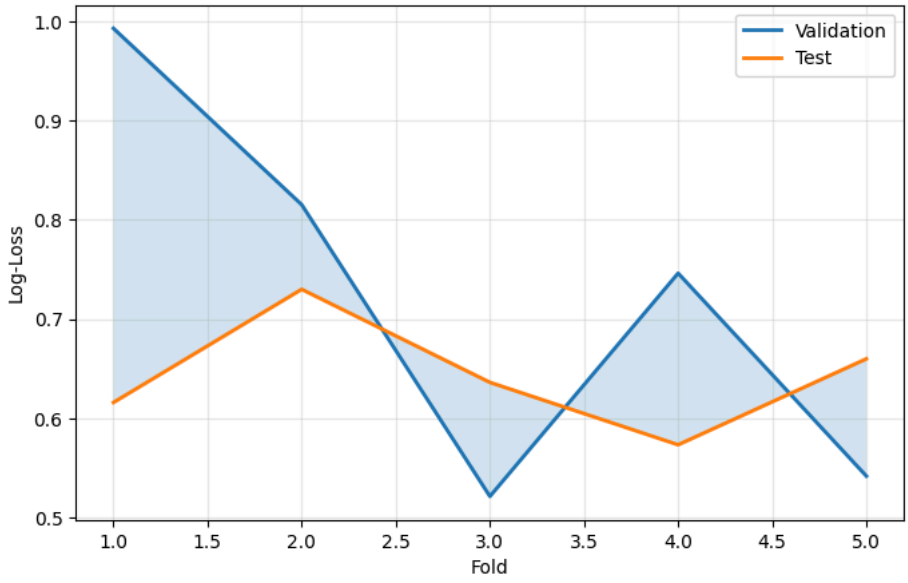



```
plt.figure(figsize=(8,5))

plt.plot(x, loss_df["val_log_loss"], linewidth=2, label="Validation")
plt.plot(x, loss_df["test_log_loss"], linewidth=2, label="Test")

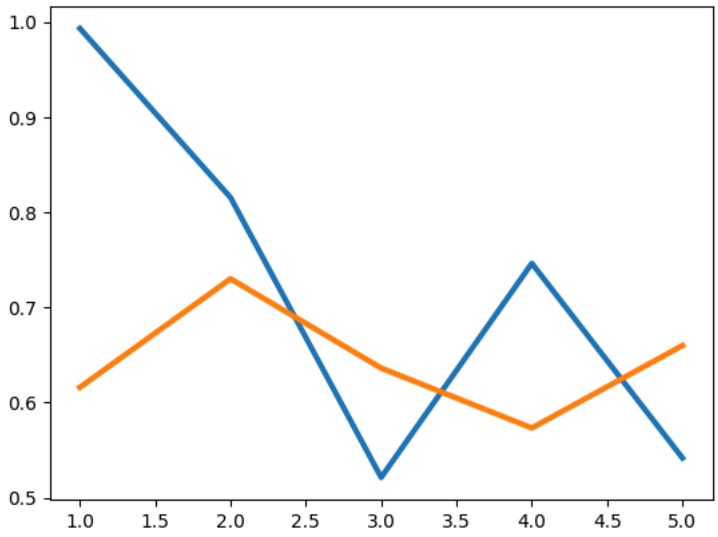
plt.fill_between(x,
                 loss_df["val_log_loss"],
                 loss_df["test_log_loss"],
                 alpha=0.2)

plt.xlabel("Fold")
plt.ylabel("Log-Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```




```
plt.plot(x, loss_df["val_log_loss"], linewidth=3)
plt.plot(x, loss_df["test_log_loss"], linewidth=3)
```

```
[<matplotlib.lines.Line2D at 0x79725e5784d0>]
```



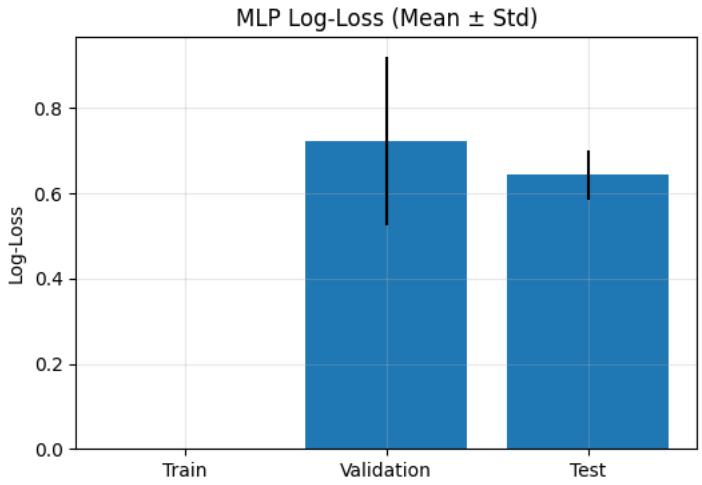

```
means = [
    loss_df["train_log_loss"].mean(),
    loss_df["val_log_loss"].mean(),
    loss_df["test_log_loss"].mean()
]

stds = [
    loss_df["train_log_loss"].std(),
    loss_df["val_log_loss"].std(),
    loss_df["test_log_loss"].std()
]

labels = ["Train", "Validation", "Test"]

plt.figure(figsize=(6,4))
plt.bar(labels, means, yerr=stds)

plt.ylabel("Log-Loss")
plt.title("MLP Log-Loss (Mean ± Std)")
plt.grid(alpha=0.3)
plt.show()
```




```
from scipy.interpolate import make_interp_spline

x_smooth = np.linspace(x.min(), x.max(), 100)

y_smooth = make_interp_spline(x, loss_df["val_log_loss"])(x_smooth)

plt.plot(x_smooth, y_smooth, linewidth=3)
```